

Reports & Analytics Module

Frontend API Implementation Guide

Date: 2026-09-21

Base path: `/api/v1/reports`

Source of truth: the actual backend code — `ReportController`, `AttendanceReportRequest`, `LeaveReportRequest`, `AttendanceReportPageResponse`, `LeaveReportPageResponse`, `AttendanceSummaryResponse`, `LeaveSummaryResponse`, `AttendanceReportServiceImpl`, `LeaveReportServiceImpl`, `AttendanceReportSpecification`, `LeaveReportSpecification`, the four exporters in `report.export`, `GlobalExceptionHandler`, `ApiError`, `WebConfig`, `SecurityConfig`, `CustomUserDetailsService` and `StringToEnumConverterFactory`.

Nothing below is invented. Anything the backend does **not** support is explicitly marked **"Not supported in backend"**.

1. What this module is

Two **read-only** endpoints. Each one serves **three formats** through a single `format` query param: paginated JSON, an `.xlsx` download, or a `.pdf` download.

Report	Endpoint	JSON	Excel	PDF
Attendance	<code>GET /api/v1/reports/attendance</code>	<code>format=json</code> (default)	<code>format=excel</code>	<code>format=pdf</code>
Leave	<code>GET /api/v1/reports/leave</code>	<code>format=json</code> (default)	<code>format=excel</code>	<code>format=pdf</code>

There is no create/update/delete, no saved report definitions, no scheduled/emailed reports, no CSV, and no employee self-service view. Both endpoints are search/filter/export only.

Not supported in backend (do not design UI for these)

- No per-employee monthly roll-up grid (`EMP x day` matrix) and no "one row per employee" aggregation – the JSON `content` is **one row per person-day** (attendance) / **one row per leave request** (leave).
- No `employeeCode` filter, no designation/job-title filter, no leave-type filter (the field exists in the request DTO but is commented out in both the DTO and the specification).
- No `search / q` free-text param – employee search is a dedicated `employeeName` param only.
- No multi-status / multi-department filter (single value only).
- No CSV export, no "all data" export cap control, no request-ID / pagination cursor.
- No sorting by employee name or department (see §11.2 – it returns a 500).

2. Global conventions

2.1 The success response is **NOT** wrapped – this module is the exception

Unlike the master/settings modules, `ReportController` returns the page DTO directly – there is **no** `ApiResponse<T>` **envelope** and no `ResponseBodyAdvice` in the project.

```
{
  "content": [ /* ... */ ],
  "summary": { /* ... */ },
  "page": 0,
  "size": 20,
  "totalElements": 150,
  "totalPages": 8,
  "first": true,
  "last": false,
  "hasNext": true,
  "hasPrevious": false
}
```

Errors, on the other hand, **are** wrapped in `ApiError` (§2.3). So one screen handles two different shapes:

Outcome	Body shape
200, <code>format=json</code>	<code>AttendanceReportPageResponse</code> / <code>LeaveReportPageResponse</code> (raw)

Outcome	Body shape
200, <code>format=excel / pdf</code>	<code>raw binary(byte[])</code>
4xx / 5xx	<code>ApiError ({ success, message, errorCode, path, timestamp })</code>

 **Important:** Do **not** write `res.data.data.content`. It is `res.data.content`. If you reuse a generic axios wrapper written for the master module, it will break here.

2.2 Pagination envelope

Both reports return the same hand-built envelope (it does **not** reuse the shared `PageResponse<T>` used elsewhere):

Field	Type	Meaning
<code>content</code>	array	current page rows
<code>summary</code>	object	statistics for the whole filtered result set , not the page (§6)
<code>page</code>	int	current page, 0-based
<code>size</code>	int	page size used
<code>totalElements</code>	long	total rows matching the filters (person-days for attendance)
<code>totalPages</code>	int	total pages
<code>first / last</code>	bool	convenience flags
<code>hasNext / hasPrevious</code>	bool	convenience flags

2.3 Error envelope (`ApiError`)

```
{
  "success": false,
  "message": "Invalid value 'XYZ' for parameter 'attendanceStatus'",
  "errorCode": "INVALID_REQUEST",
  "path": "/api/v1/reports/attendance",
```

```
"timestamp": "2026-09-21T11:04:22.918233"
}
```

Codes you will actually see in this module:

HTTP	errorCode	Trigger	Message
400	INVALID_REQUEST	bad scalar type, bad enum value, bad date format	Invalid value 'XYZ' for parameter 'attendanceStatus'
401	UNAUTHORIZED	missing/expired token (<code>JwtAuthenticationEntryPoint</code>)	Authentication is required to access this resource.
403	ACCESS_DENIED	role not allowed (<code>JwtAccessDeniedHandler</code>)	You do not have permission to perform this action.
500	INTERNAL_SERVER_ERROR	bad <code>sortBy</code> , bad <code>sortDir</code> , date-range validation, export failure, anything unhandled	An unexpected error occurred. (<code><ExceptionSimpleName></code> : <code><message></code>)

`ApiError` has **no** `errors[]` **array** – only one message, ever.

Note the 500 row. `AttendanceReportServiceImpl.validateRequest / LeaveReportServiceImpl.validateLeaveRequest` throw a raw `IllegalArgumentException` ("Start date cannot be after end date."), and the service sort (`Sort.Direction.fromString`) throws the same type. Neither is mapped, so they fall into the catch-all `@ExceptionHandler(Exception.class)` → **500**, not 400. Treat 500 as "the UI sent something invalid", not as a server outage.

2.4 Authentication & roles – frontend role requirement

- Bearer JWT is required on both endpoints: `Authorization: Bearer <accessToken>`.
- **Frontend roles for this module:** `HR_ADMIN` and `MANAGER`.
- The Reports menu, pages, filters, and export actions should be visible only to users with `HR_ADMIN` or `MANAGER`.

- Backend authorization must also allow `HR_ADMIN` and `MANAGER`. The source code currently contains an `HR` vs `HR_ADMIN` mismatch; if the backend has not yet been updated, `HR_ADMIN` will receive 403 until the authorization expression is changed to `hasAnyRole('HR_ADMIN', 'MANAGER')`.
- Other roles — `SUPER_ADMIN`, `EMPLOYEE`, `PAYROLL_ADMIN`, and `CLIENT` — should not be given access to the Reports module.

2.5 CORS

`WebConfig` allows any origin with `allowedOrigins("*")`, `allowedMethods("*")`, `allowedHeaders("*")` and **no** `exposedHeaders`. Practical effects:

- Cross-origin calls work from `localhost:3000`, etc.
- The browser **cannot read** `Content-Disposition` on the file downloads, so you cannot recover the server's filename — set the filename yourself (§7).
- No cookies/credentials are used, so `Authorization` must be set by your HTTP client on every call.

2.6 Enum query params are case-insensitive, and some values silently mean "no filter"

`StringToEnumConverterFactory` is registered globally, so `attendanceStatus` / `leaveStatus` accept any casing (`present`, `Present`, `PRESENT`).

It also maps these values to `null` (filter silently ignored):

```
" "  "--"  "-"  "all"  "none"  "null"  "select"      (case-insensitive, trimmed)
```

This is **safe and useful** — sending `attendanceStatus=` or `attendanceStatus=all` disables the filter instead of erroring. But any *other* unrecognised value throws → **400 INVALID_REQUEST** (`Invalid value 'Foo' for parameter 'attendanceStatus'`).

`format` is a plain string, not an enum. It is lower-cased server-side, and **anything that is not excel or pdf falls through to JSON** — `format=csv` returns a normal JSON page with status 200. Guard the export buttons client-side; don't rely on the server to reject `csv`.

2.7 Date/time formats

- Request dates: `YYYY-MM-DD ( @DateTimeFormat(iso = DATE) )`. Anything else → 400
`Invalid request format`.
- Response dates: `attendanceDate, startDate, endDate` → `yyyy-MM-dd`.
- Response timestamps: `checkInTime, checkOutTime, createdAt, updatedAt` → `yyyy-MM-dd'T'HH:mm:ss` — **no timezone offset and no milliseconds** (`@JsonFormat` on the DTOs; `WRITE_DATES_AS_TIMESTAMPS` is disabled in `JacksonConfig`).

Because there is no offset, the values are true local wall-clock times. `new Date("2026-07-27T09:00:00")` in JS parses them as local time, which matches the server's intent. Do **not** run them through a UTC conversion before display or you will shift every check-in time. Simplest safe options: display with `date-fns` / `dayjs` formatting, or string-slice the `HH:mm` part.

3. Endpoint summary

#	Method	Path	Roles	Body
1	GET	<code>/api/v1/reports/attendance</code>	<code>MANAGER</code> (see §2.4)	Attendance page / xlsx / pdf
2	GET	<code>/api/v1/reports/leave</code>	<code>MANAGER</code> (see §2.4)	Leave page / xlsx / pdf

Both are `@GetMapping` with **all parameters as query params** (no path variables, no request body). Both service classes are `@Transactional(readOnly = true)`.

4. Attendance report

GET `/api/v1/reports/attendance`

4.1 Query parameters

Param	Type	Default	Validation	Notes
<code>format</code>	string	<code>json</code>	none	<code>json</code> <code>excel</code> <code>pdf</code> . Unknown → JSON.

Param	Type	Default	Validation	Notes
<code>employeeId</code>	long	-	none	exact match on <code>attendance.employee.id</code>
<code>employeeName</code>	string	-	<code>@Size(max=100)</code> (DTO only, see §11.1)	partial, case-insensitive, matches first name OR last name (§11.4)
<code>departmentId</code>	long	-	none	exact match on <code>employee.department.id</code>
<code>attendanceStatus</code>	enum	-	case-insensitive; <code>all / none / " → no filter</code>	<code>PRESENT, LATE, HALF_DAY, ABSENT, LEAVE, HOLIDAY, WEEKEND, MISSED_CHECKOUT</code>
<code>startDate</code>	date	-	ISO <code>YYYY-MM-DD</code>	inclusive lower bound on <code>attendanceDate</code>
<code>endDate</code>	date	-	ISO <code>YYYY-MM-DD</code>	inclusive upper bound on <code>attendanceDate</code>
<code>month</code>	int	-	<code>@Min(1)</code> <code>@Max(12)</code>	requires <code>year</code> ; overwrites <code>startDate / endDate</code> (§8.1)
<code>year</code>	int	-	<code>@Min(2000)</code> <code>@Max(2100)</code>	requires <code>month</code>
<code>page</code>	int	0	<code>@Min(0)</code>	0-based; JSON only
<code>size</code>	int	20	<code>@Min(1)</code> <code>@Max(100)</code>	JSON only; <code>@Max(100)</code> caps it
<code>sortBy</code>	string	<code>attendanceDate</code>	see §11.2	JPA entity property name , not a response field name
<code>sortDir</code>	string	<code>DESC</code>	see §11.2	<code>ASC / DESC</code> , case-insensitive

Sort fields that work on the Attendance entity: `attendanceDate` , `checkInTime` , `checkOutTime` , `workingMinutes` , `totalBreakMinutes` , `attendanceStatus` , `lateMinutes` , `earlyExitMinutes` , `overtimeMinutes` , `id` .

Sort fields that 500: `employeeName` , `departmentName` , `workingHours` , `breakMinutes` , `employeeCode` , `employeeId` — despite what the Swagger annotations suggest. Dotted paths such as `employee.firstName` are resolved by Spring Data JPA and are the only server-side way to sort by name (verify once in your environment).

4.2 Response — `AttendanceReportPageResponse`

```
{
  "content": [
    {
      "employeeId": 12,
      "employeeCode": "EMP0012",
      "employeeName": "John Doe",
      "departmentName": "Engineering",
      "attendanceDate": "2026-07-27",
      "checkInTime": "2026-07-27T09:04:11",
      "checkOutTime": "2026-07-27T18:12:03",
      "workingMinutes": 488,
      "breakMinutes": 60,
      "attendanceStatus": "PRESENT",
      "workingHours": 8.133333333333333
    }
  ],
  "summary": {
    "totalRecords": 150,
    "presentCount": 118,
    "absentCount": 6,
    "lateCount": 9,
    "halfDayCount": 3,
    "leaveCount": 8,
    "holidayCount": 2,
    "weekendCount": 4,
    "attendancePercentage": 78.66666666666666
  },
  "page": 0,
  "size": 20,
  "totalElements": 150,
  "totalPages": 8,
  "first": true,
  "last": false,
  "hasNext": true,
```

```
"hasPrevious": false
}
```

Row field semantics (all from `AttendanceReportServiceImpl.mapToResponse`):

Field	Notes
<code>employeeName</code>	<code>firstName + " " + lastName</code> – may have a trailing space if <code>lastName</code> is null
<code>departmentName</code>	<code>employee.department.departmentName</code> (department is <code>NOT NULL</code> on employee, so always present)
<code>checkInTime</code> / <code>checkOutTime</code>	<code>null</code> for ABSENT / LEAVE / HOLIDAY / WEEKEND rows – render <code>–</code> , not "Invalid Date"
<code>workingMinutes</code>	total minutes; <code>0</code> for leave rows, may be <code>null</code> in theory (nullable column)
<code>breakMinutes</code>	this is <code>attendance.totalBreakMinutes</code> – not the same field name as the entity
<code>workingHours</code>	<code>workingMinutes / 60.0</code> , unrounded(<code>8.133333333333333</code>); <code>null</code> when <code>workingMinutes</code> is null. Format client-side to 2 decimals or <code>Xh Ym</code>
<code>attendanceStatus</code>	enum name, one of the 8 values above

The response does **not** expose `employeeStatus`, coordinates, `lateMinutes`, `earlyExitMinutes`, or overtime minutes, even though the entity stores them.

5. Leave report

GET `/api/v1/reports/leave`

5.1 Query parameters

Param	Type	Default	Notes
<code>format</code>	string	<code>json</code>	same rules as attendance
<code>employeeId</code>	long	–	exact match
<code>employeeName</code>	string	–	partial, case-insensitive, first OR last name
<code>departmentId</code>	long	–	exact match
<code>leaveStatus</code>	enum	–	<code>PENDING, APPROVED, REJECTED, CANCELLED</code> ; unknown → 400; <code>all / none /</code> → no filter
<code>startDate</code>	date	–	inclusive lower bound on the leave's <code>startDate</code>
<code>endDate</code>	date	–	inclusive upper bound on the leave's <code>endDate</code> – see §8.2, this is <i>containment</i> , not overlap
<code>month</code>	int	–	requires <code>year</code> ; overwrites the dates
<code>year</code>	int	–	requires <code>month</code>
<code>page</code>	int	<code>0</code>	0-based
<code>size</code>	int	<code>20</code>	max 100
<code>sortBy</code>	string	<code>createdAt</code>	property names: <code>createdAt, updatedAt, startDate, endDate, totalDays, reason, status, id</code>
<code>sortDir</code>	string	<code>DESC</code>	<code>ASC / DESC</code>

Sort fields that 500: `employeeName, departmentName, leaveStatus, leaveType, employeeCode, employeeId`. Note it is `status` (the entity property), **not** `leaveStatus` (the response field), even though the Swagger example says `leaveStatus`.

5.2 Response – `LeaveReportPageResponse`

```

{
  "content": [
    {
      "employeeId": 12,
      "employeeCode": "EMP0012",
      "employeeName": "John Doe",
      "departmentName": "Engineering",
      "leaveId": 341,
      "leaveType": "Casual Leave",
      "leaveStatus": "APPROVED",
      "startDate": "2026-07-28",
      "endDate": "2026-07-30",
      "totalDays": 3,
      "reason": "Family function",
      "createdAt": "2026-07-22T10:31:00",
      "updatedAt": "2026-07-23T09:02:41"
    }
  ],
  "summary": {
    "totalLeaves": 75,
    "approvedLeaves": 60,
    "pendingLeaves": 10,
    "rejectedLeaves": 3,
    "cancelledLeaves": 2,
    "totalLeaveDays": 225,
    "averageLeaveDays": 3.0,
    "uniqueEmployees": 45
  },
  "page": 0, "size": 20, "totalElements": 75, "totalPages": 4,
  "first": true, "last": false, "hasNext": true, "hasPrevious": false
}

```

- `leaveId` is the **leave request id** (usable as a row key and for deep-linking into the leave detail screen).
- `leaveType` is a **plain string** taken from `leave_type.name` in the database — not an enum. Seeds in this codebase are `Sick Leave` and `Casual Leave` (the seeder is currently disabled, so treat the values as data, not constants). Never branch UI logic on this string.
- `createdAt` / `updatedAt` come from the shared `BaseEntity` auditing columns; `updatedAt` is the last time the request row changed (status change, etc.).

6. Summary semantics (exact rules – read before labelling the cards)

The summary is computed by a **second, unpaginated query** over the entire filtered result set (`attendanceRepository.findAll(specification)`). So:

- `summary` **never changes when the user changes page** – it describes the whole filtered range.
- It is **not** page-scoped. Do not render it as "this page" or "page totals".
- It is expensive: every summary call loads all matching entities into memory. A one-year, all-employees attendance report = every person-day row in JVM memory, twice (once paged, once for the summary). Keep the default date range tight (the UI should default to the current month).

Attendance

```
totalRecords          = number of attendance ROWS matching the filters
(person-days, not employees)
presentCount          = status == PRESENT
absentCount           = status == ABSENT
lateCount             = status == LATE
halfDayCount          = status == HALF_DAY
leaveCount            = status == LEAVE
holidayCount          = status == HOLIDAY
weekendCount          = status == WEEKEND
attendancePercentage = presentCount / totalRecords * 100      (0 when
totalRecords == 0)
```

Two things the UI must not get wrong:

1. `attendancePercentage` **is a row-level percentage, not a per-employee attendance rate.**
Filtering one employee for July gives "present rows / all rows in July", where LATE, HALF_DAY and leave rows all count in the denominator but not the numerator. Label it "*Present-day share of records*" rather than "Attendance %", or compute your own rate.
2. **The status counts do not always sum to `totalRecords`.** `MISSED_CHECKOUT` exists in the enum but has no counter in the summary. Never render a pie/donut of the status counts as if it were exhaustive; if you need completeness, plot `totalRecords - (sum of counters)` as "Other".

Leave

```
totalLeaves      = number of leave requests matching the filters
pendingLeaves    = status == PENDING
approvedLeaves   = status == APPROVED          (only APPROVED – not
MANAGER_APPROVED/HR_APPROVED)
rejectedLeaves   = status == REJECTED
cancelledLeaves  = status == CANCELLED
totalLeaveDays    = SUM(totalDays) over ALL matched requests, every status
averageLeaveDays  = totalLeaveDays / totalLeaves  (0 when totalLeaves ==
0)
uniqueEmployees  = COUNT(DISTINCT employeeId) in the matched set
```

- `pending + approved + rejected + cancelled == totalLeaves` holds for leave (only four statuses exist).
 - `totalLeaveDays` **includes rejected, cancelled and pending requests**. It is "days requested in this range", not "days actually taken". Label it accordingly, or rebuild the figure from the rows you already have.
 - `uniqueEmployees` counts distinct employees inside the filtered range – a useful "how many people took leave" card, but note it counts anyone with a matched row, including employees whose only row is a rejected request.
-

7. File exports (Excel & PDF)

`format=excel` / `format=pdf` short-circuit the pagination entirely:

- `page` and `size` are **ignored**; every row matching the filters is exported.
- `sortBy` / `sortDir` **are** applied to the export query, so the file ordering matches the table ordering.
- Response headers: `Content-Disposition: attachment; filename=attendance-report.xlsx` (fixed name, no date or filter info) and `Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`, or `application/pdf`.

Excel column order

Attendance (`AttendanceExcelExporter`): `Employee Code`, `Employee Name`, `Department`, `Attendance Date`, `Check In`, `Check Out`, `Working Minutes`, `Break Minutes`, `Status` (9 columns; empty strings where check-in/out are null).

Leave(`LeaveExcelExporter`): `Employee Code`, `Employee Name`, `Department`, `Leave Type`, `Leave Status`, `Start Date`, `End Date`, `Total Days`, `Reason`, `Created At`, `Updated At`, `EmployeeId`, `Leave Request Id` (13 columns).

PDF column order

Attendance(`AttendancePdfExporter`): same 9 columns as the Excel, landscape A4.

Leave(`LeavePdfExporter`): **different order from the Excel** — `Employee Code`, `Employee Id`, `Employee Name`, `Department`, `Leave Type`, `Leave Id`, `Status`, `Start Date`, `End Date`, `Days`, `Reason`, `Created At` (12 columns, `Updated At` omitted, `Employee Id` moved to second).

Neither exporter emits a title row with the applied filters, a date range, a generation timestamp, a totals row, or page numbers. If the business needs "exported for July 2026, Engineering", you must add it backend-side.

A robustness note: `AttendanceExcelExporter` unboxes `workingMinutes` / `breakMinutes` into a numeric cell, so a single row with `null` in either column aborts the whole export with a 500 (the PDF exporter would instead print the literal string `null`). All current write paths (check-in, leave marking, seeder) set `0` or a computed value, so this is latent rather than active — but if you ever see "Failed to generate attendance excel report", look for null working/break minutes first.

8. Filter interaction rules (the part that causes bug reports)

8.1 `month + year` silently override `startDate / endDate`

If both `month` and `year` are present, the service replaces `startDate / endDate` with the first and last day of that month **before** building the query:

```
firstDay = LocalDate.of(year, month, 1);
lastDay  = firstDay.withDayOfMonth(firstDay.lengthOfMonth());
```

So a request with `month=7&year=2026&startDate=2026-07-10&endDate=2026-07-20` returns the **whole of July**. The UI must send **either** `month + year` **or** `startDate + endDate`, never both.

Also: if `month` is present but `year` is **not**, the month is silently ignored and the request returns **all data**. Never allow the UI to submit a month without a year (make the year selector mandatory, or default it to the current year).

8.2 Date range means different things per report

	Attendance	Leave
Lower bound on	<code>attendanceDate >= startDate</code>	<code>startDate >= startDate</code>
Upper bound on	<code>attendanceDate <= endDate</code>	<code>endDate <= endDate</code>
Effect	day-by-day rows inside the window	the leave must be fully contained in the window

So for the leave report, a request that runs **28 Jun** → **3 Jul** will **not** appear in a "July" query: it starts before the window and ends after it starts. A leave from 29–31 July appears, but 28 Jul → 2 Aug does not.

For an "all leave touching this month" view, widen the range client-side (e.g. `startDate = monthStart - 45d`, `endDate = monthEnd + 45d`) and label the column set honestly, or ask the backend for overlap semantics (`startDate <= rangeEnd AND endDate >= rangeStart`). Do not silently show an incomplete list.

8.3 Employee name search

- Case-insensitive `LIKE %term%` on **first name OR last name**.
- A **full name does not match**: `employeeName=John Doe` returns nothing, because neither `first_name` nor `last_name` contains `"john doe"`. Search one token, or make your own picker send `employeeId` instead (recommended – the row already carries `employeeId`).
- The term is **not trimmed**: `"John "` (trailing space) → `LIKE '%john %'` → no rows. Trim before sending.
- An empty string is ignored (`isBlank()` check), so sending `employeeName=` is safe.
- There is no `employeeCode` search, even though the response carries `employeeCode`.

8.4 Blank enum handling is a feature, use it

Because `""`, `all`, `none` and `null` all convert to "no filter" (§2.6), a dropdown can keep a `"- All statuses -"` option that submits `attendanceStatus=all`, or simply omit the param. Either is fine; the difference is only cosmetic in your URL state.

9. Recommended frontend flows

9.1 Screens

1. **Reports hub** with two tabs: *Attendance* and *Leave*.
2. Each tab = **filter bar** (department, employee, status, period) + **summary cards** + **paged table** + **Export Excel / Export PDF** buttons.
3. There is no detail screen for attendance rows; a leave row can deep-link into the existing leave-request detail screen using `leaveId`.

9.2 Filter bar

- **Department dropdown** → `GET /api/v1/lookups/departments` (any authenticated user; returns id + name).
- **Employee picker** → `GET /api/v1/lookups/employee-id-name` (id, `employeeCode`, `employeeName`) for a client-side-filtered searchable select. This avoids the "full name doesn't match" trap from §8.3 and gives you `employeeId` directly.
- **Status dropdown** → local constants (the 8 attendance statuses / 4 leave statuses). No backend endpoint needed.
- **Period control** → a single mode toggle between "Month" (`month` + `year`) and "Custom range" (`startDate` + `endDate`). Never send both.
- ⚠ `/api/v1/lookups/**` responses **are** wrapped in `ApiResponse ( { success, message, data, timestamp } )`, unlike the report endpoints. Use the right unwrapping for each call.
- Debounce `employeeName` typing by ~350 ms and always reset `page` to 0 when any filter changes.

9.3 State & URL

Keep the filter object as the single source of truth, serialise it into the URL query string so a report is shareable/bookmarkable, and derive the request params from it exactly once (§12.2). Suggested defaults: `month + year` = current month, `page=0`, `size=20`, `sortBy=attendanceDate|createdAt`, `sortDir=DESC`.

9.4 Export buttons

- Build the export query from the **same filter object** but drop `page / size` and set `format`.
- Disable the button while a request is in flight and show a spinner — a year-long export can take a while (no streaming, the whole file is built in memory).
- Set the filename client-side (`attendance-report-2026-07.xlsx`) because `Content-Disposition` is not readable cross-origin.
- Handle errors in blob mode: a failed export still returns the `ApiError` **as a Blob**, so you must read it as text and parse the JSON to show `message` (§12.4).

9.5 Role gating

Until the backend `@PreAuthorize` is fixed (§2.4), gate the entire Reports nav on `role === 'MANAGER'`. If you expose it to `HR_ADMIN`, the screen will render a 403 with the generic "You do not have permission" message.

10. Error handling cheat sheet

Symptom	Cause	What the UI should do
400 <code>INVALID_REQUEST</code> Invalid value 'x' for parameter 'attendanceStatus'	typo'd enum value	bug in your status dropdown; log it
400 <code>INVALID_REQUEST</code> Invalid request format	<code>startDate=27-07-2026</code> etc.	always serialise dates as <code>YYYY-MM-DD</code>
401 <code>UNAUTHORIZED</code>	expired/missing access token	refresh token, retry once, then logout
403 <code>ACCESS_DENIED</code>	role not in <code>HR / MANAGER</code> (i.e. anyone but <code>MANAGER</code>)	hide the feature; don't retry
500 with <code>IllegalArgumentException: Start date cannot be after end date</code>	inverted range	validate client-side before calling

Symptom	Cause	What the UI should do
500 with <code>IllegalArgumentException: Unable to locate Attribute with the given name [employeeName]</code>	invalid <code>sortBy</code>	restrict <code>sortBy</code> to the verified list (§4.1/§5.1)
500 with <code>Invalid value 'X' for parameter 'size' (method- validation)</code>	<code>size / month / year / p age</code> outside its <code>@Min / @Max</code>	clamp client-side: <code>size</code> 1- 100, <code>page</code> <code>>= 0</code> , <code>month</code> 1-12, <code>year</code> 2000-2100
200 but the "export" downloaded a JSON file	<code>format</code> was not <code>excel / pdf</code>	only ever send the three known values

The range-constraint case is the least obvious: the constraints are declared on the controller parameters and enforced by class-level `@Validated`, but no `@ExceptionHandler` covers the resulting method-validation exception, so it lands in the catch-all 500. Clamp values in the UI rather than relying on a 400.

12. TypeScript implementation

12.1 Types

```
// ----- enums (mirror the backend exactly) -----
export type AttendanceStatus =
  | 'PRESENT' | 'LATE' | 'HALF_DAY' | 'ABSENT'
  | 'LEAVE' | 'HOLIDAY' | 'WEEKEND' | 'MISSED_CHECKOUT';

export type LeaveStatus = 'PENDING' | 'APPROVED' | 'REJECTED' | 'CANCELLED';

// ----- attendance report -----
export interface AttendanceReportRow {
  employeeId: number;
  employeeCode: string;
  employeeName: string;           // "First Last" (may end with a space)
  departmentName: string;
  attendanceDate: string;         // YYYY-MM-DD
  checkInTime: string | null;     // YYYY-MM-DDTHH:mm:ss (local, no offset)
```

```
    checkOutTime: string | null;
    workingMinutes: number | null;
    breakMinutes: number | null;
    attendanceStatus: AttendanceStatus;
    workingHours: number | null;
}
```

```
export interface AttendanceSummary {
    totalRecords: number;
    presentCount: number;
    absentCount: number;
    lateCount: number;
    halfDayCount: number;
    leaveCount: number;
    holidayCount: number;
    weekendCount: number;
    attendancePercentage: number;
}
```

```
export interface AttendanceReportPage {
    content: AttendanceReportRow[];
    summary: AttendanceSummary;
    page: number;
    size: number;
    totalElements: number;
    totalPages: number;
    first: boolean;
    last: boolean;
    hasNext: boolean;
    hasPrevious: boolean;
}
```

```
// ----- leave report -----
```

```
export interface LeaveReportRow {
    employeeId: number;
    employeeCode: string;
    employeeName: string;
    departmentName: string;
    leaveId: number;
    leaveType: string;           // free text from DB, e.g. "Casual Leave"
    leaveStatus: LeaveStatus;
    startDate: string;          // YYYY-MM-DD
    endDate: string;            // YYYY-MM-DD
    totalDays: number;
}
```

```
    reason: string;
    createdAt: string;           // YYYY-MM-DDTHH:mm:ss
    updatedAt: string | null;
}
```

```
export interface LeaveSummary {
    totalLeaves: number;
    approvedLeaves: number;
    pendingLeaves: number;
    rejectedLeaves: number;
    cancelledLeaves: number;
    totalLeaveDays: number;
    averageLeaveDays: number;
    uniqueEmployees: number;
}
```

```
export interface LeaveReportPage {
    content: LeaveReportRow[];
    summary: LeaveSummary;
    page: number;
    size: number;
    totalElements: number;
    totalPages: number;
    first: boolean;
    last: boolean;
    hasNext: boolean;
    hasPrevious: boolean;
}
```

```
// ----- errors (unchanged across the app) -----
export interface ApiError {
    success: false;
    message: string;
    errorCode: string;
    path: string;
    timestamp: string;
}
```

```
// ----- safe sort allowlists -----
export const ATTENDANCE_SORT_FIELDS = [
    'attendanceDate', 'checkInTime', 'checkOutTime',
    'workingMinutes', 'totalBreakMinutes', 'attendanceStatus', 'id',
] as const;
```

```
export const LEAVE_SORT_FIELDS = [
  'createdAt', 'updatedAt', 'startDate', 'endDate', 'totalDays', 'status'
] as const;

export type AttendanceSortField = typeof ATTENDANCE_SORT_FIELDS[number];
export type LeaveSortField = typeof LEAVE_SORT_FIELDS[number];
```

12.2 Filter state → query string

```
export type PeriodMode = 'month' | 'range';

export interface ReportFilters {
  departmentId?: number;
  employeeId?: number;
  employeeName?: string;
  status?: AttendanceStatus | LeaveStatus; // one field, mapped per rep
  periodMode: PeriodMode;
  month?: number; // used only when periodMode === 'month'
  year?: number;
  startDate?: string; // used only when periodMode === 'range'
  endDate?: string;
  page?: number;
  size?: number;
  sortBy?: string;
  sortDir?: 'ASC' | 'DESC';
}

function toQuery(filters: ReportFilters, format: 'json' | 'excel' | 'pdf') {
  const q = new URLSearchParams();

  q.set('format', format);

  if (filters.departmentId != null) q.set('departmentId', String(filters.departmentId));
  if (filters.employeeId != null) q.set('employeeId', String(filters.employeeId));

  const name = filters.employeeName?.trim(); // backend does NOT
  if (name) q.set('employeeName', name);

  // §8.1 – never send month/year together with an explicit date range
  if (filters.periodMode === 'month') {
    if (filters.month != null && filters.year != null) { // year is mandatory
      q.set('month', String(filters.month));
      q.set('year', String(filters.year));
    }
  }
}
```

```

    }
  } else {
    if (filters.startDate) q.set('startDate', filters.startDate);
    if (filters.endDate) q.set('endDate', filters.endDate);
  }

  if (filters.status) q.set('statusParamName', String(filters.status)); /

  // §11.2 / §11.7 – allowlisted sort field, clamped page/size
  if (filters.sortBy) q.set('sortBy', filters.sortBy);
  q.set('sortDir', filters.sortDir === 'ASC' ? 'ASC' : 'DESC');

  if (format === 'json') {
    q.set('page', String(Math.max(0, filters.page ?? 0)));
    q.set('size', String(Math.min(100, Math.max(1, filters.size ?? 20))))
  }
  return q;
}

```

In practice, build the query with the correct status param name per report: `attendanceStatus` for attendance, `leaveStatus` for leave. Pass the clamped `size` / `page` **only** for `format=json` – they are ignored for exports.

12.3 Fetching a JSON page

```

const BASE = process.env.NEXT_PUBLIC_API_URL ?? 'http://localhost:8080';

export async function getAttendanceReport(
  filters: ReportFilters,
  token: string,
): Promise<AttendanceReportPage> {
  const q = toQuery(filters, 'json');
  const res = await fetch(`${BASE}/api/v1/reports/attendance?${q}`, {
    headers: { Authorization: `Bearer ${token}` },
  });

  if (!res.ok) {
    const err: ApiError = await res.json(); // error body IS an ApiE
    throw new Error(err.message);
  }
  return res.json(); // success body is NOT
}

```

```

export async function getLeaveReport(
  filters: ReportFilters,
  token: string,
): Promise<LeaveReportPage> {
  const q = toQuery(filters, 'json');
  q.set('leaveStatus', String(filters.status ?? ''));
  q.delete('statusParamName');
  const res = await fetch(`${BASE}/api/v1/reports/leave?${q}`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  if (!res.ok) throw new Error(((await res.json()) as ApiError).message);
  return res.json();
}

```

12.4 Blob download (Excel / PDF)

```

export async function downloadReport(
  kind: 'attendance' | 'leave',
  filters: ReportFilters,
  format: 'excel' | 'pdf',
  token: string,
) {
  const q = toQuery(filters, format); // page/size omitted c
  if (kind === 'leave') {
    q.set('leaveStatus', String(filters.status ?? ''));
    q.delete('statusParamName');
  }

  const res = await fetch(`${BASE}/api/v1/reports/${kind}?${q}`, {
    headers: { Authorization: `Bearer ${token}` },
  });

  if (!res.ok) {
    // In this mode Spring still returns the ApiError JSON – but as a str
    const text = await res.text();
    let message = `Export failed (HTTP ${res.status})`;
    try { message = (JSON.parse(text) as ApiError).message; } catch { /*
    throw new Error(message);
  }

  const blob = await res.blob();
  // Content-Disposition is NOT readable cross-origin (no exposedHeaders)

```

```

const ext = format === 'excel' ? 'xlsx' : 'pdf';
const stamp = filters.periodMode === 'month'
  ? `${filters.year}-${String(filters.month).padStart(2, '0')}`
  : `${filters.startDate ?? 'all'}_${filters.endDate ?? 'all'}`;

const url = URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = url;
a.download = `${kind}-report-${stamp}.${ext}`;
document.body.appendChild(a);
a.click();
a.remove();
URL.revokeObjectURL(url);
}

```

axios equivalent (mind `responseType`):

```

const res = await api.get(`/api/v1/reports/attendance`, {
  params,
  responseType: 'blob', // required for excel/pdf
  headers: { Authorization: `Bearer ${token}` },
});
// errors arrive as a Blob too:
const message = res.status >= 400
  ? JSON.parse(await res.data.text()).message
  : null;

```

12.5 Formatting helpers

```

// workingHours is unrounded (8.133333333333333); prefer minutes.
export const formatMinutes = (m?: number | null) =>
  m == null ? '-' : `${Math.floor(m / 60)}h ${String(m % 60).padStart(2,

// "2026-07-27T09:04:11" -> "09:04" (no timezone conversion: values are 1
export const timeOfDay = (iso?: string | null) =>
  iso ? iso.slice(11, 16) : '-';

export const EMPTY = '-'; // null check-in/out and null working minutes

```

Frontend rules to remember

1. Roles: Reports are for `HR_ADMIN` and `MANAGER` .

2. Response shape: Successful JSON reports are **not wrapped** in `ApiResponse` ; use `res.data.content / res.json()` .

3. Period: Send either `month + year` or `startDate + endDate` — never both.

4. Exports: `page` and `size` are ignored for Excel/PDF; the complete filtered result is exported.

5. Sorting: Send only verified JPA property names. Invalid sort fields can produce `500` .

6. Performance: Default the UI to the current month because summaries are calculated over the entire filtered result set.

13. UI validation checklist

- `size` between 1 and 100; `page` ≥ 0 .
 - `month` 1-12 **and** `year` 2000-2100, and `year` is always present when `month` is.
 - Never send `month + year` together with `startDate / endDate` (§8.1).
 - `startDate` $\leq$ `endDate` — otherwise the backend answers **500**, not 400.
 - Status values only from the two enum lists; send `'` / `all` or omit to clear.
 - `sortBy` only from the allowlists in §12.1; `sortDir` only `ASC / DESC` .
 - Trim `employeeName` ; search a single token, or use the employee picker's `employeeId` .
 - Reset `page` to 0 whenever any filter changes.
 - Default the period to the current month to keep the summary query cheap (§6).
-